

Antigr... > Headl...

Headless mode

Run Antigravity CLI non-interactively to script agent tasks, integrate with CI pipelines, and capture machine-readable output.

Headless mode (also called print mode) sends a single prompt to the agent, streams or returns the response, and exits. Use it whenever you need the agent's output in a program instead of a terminal UI.

Run a single prompt

Pass a prompt with

`-p` (or its aliases

`--print` and

`--prompt`) to run

once and exit:

```
bash   
agy -p "In  
one  
sentence,  
what is a  
git rebase?"
```

```
text   
A git rebase  
rewrites the  
commit  
history by  
transplanting  
a sequence of  
commits onto  
a new base  
commit,  
imposing a  
strictly  
linear  
progression  
of changes  
that  
eliminates  
arbitrary  
merge  
artifacts.
```

The response goes
to `stdout` .
Diagnostics —
errors,
authentication
prompts, progress,
and permission
notices — go to
`stderr` . This split
keeps the captured
response clean:

```
bash   
# Capture  
only the  
model  
response;  
diagnostics  
still print  
to the  
terminal.  
answer=$(agy  
-p "Name  
three  
popular  
version  
control  
systems,  
comma-  
separated.")
```

Note: Headless mode uses your cached credentials. Authenticate once with an interactive `agy` session first. In a non-interactive environment with no terminal (for example, CI), a run that is not already authenticated exits with an `authentication required` error instead of hanging.

Output formats

The

`--output-format`
flag controls the
shape of `stdout` .
It accepts three
values:

Format	<code>stdout</code> shape	Use it for
<code>text</code>	The response text (default)	Human- readable output, quick scripts
<code>json</code>	One JSON object printed on completion	Capturing a result plus metadata
<code>stream- json</code>	Newline- delimited JSON (NDJSON) events	Monitoring progress, tools, and token usage

Text

The default. The
response text goes
straight to `stdout`
with no wrapping:

bash



```
agy -p "In  
one  
sentence,  
what does  
the command  
git bisect  
do?"
```

text 

Git bisect
executes a
binary
search
algorithm
across a
project's
commit
history to
rapidly
isolate the
precise
commit
responsible
for
introducing
a defect.

JSON

Set

```
--output-format  
json
```

to get a single
JSON envelope
after the run
completes. The CLI
emits it on one line;
pipe through `jq`
to pretty-print:

```
bash   
agy -p "In  
one  
sentence,  
what is a  
git rebase?"  
--output-  
format json  
| jq
```

```
json 
```

```

{
  "conversation_id": "055a398f-db14-abb-1bf03f812",
  "status": "SUCCESS",
  "response": "rebase rewrite commit history transplanting sequence of commits onto a new base commit, imposing strictly linear progression of changes that eliminates arbitrary merge artifact",
  "duration_seconds": 7.16,
  "num_turns": {
    "usage": {
      "input_tokens": 10415,
      "output_tokens": 657,
      "thinking_tokens": 616,
      "cache_read_tokens": 8113,
      "total_tokens": 11072
    }
  }
}

```

The envelope contains these fields:

Field	Type	Description
<code>conversation_id</code>	string	ID of the conversation, for resuming later
<code>status</code>	string	Terminal status (see Status values)
<code>response</code>	string	The agent’s free-text response
<code>error</code>	string	Error message; present only on failure
<code>duration_seconds</code>	number	Wall-clock duration of the run
<code>num_turns</code>	number	Number of user turns in the conversation
<code>structured_output</code>	object	Parsed schema output; present only with <code>--json-schema</code>
<code>json_schema</code>	object	The schema that was enforced; present only with <code>--json-schema</code>
<code>usage</code>	object	Token counts: <code>input_tokens</code> , <code>output_tokens</code> , <code>thinking_tokens</code> , <code>cache_read_tokens</code> , <code>total_tokens</code>

Structured output with a schema

Pass

`--json-schema` to

constrain the

answer to a

schema. The parsed

object appears in

`structured_output`

, and `response`

holds the same

payload serialized

as a string:

```
bash   
agy -p "Parse  
version string  
object with in  
major, minor,  
--output-for  
--json-schem  
'{"type":"obje  
{"major":  
{"type":"integ  
{"type":"integ  
{"type":"integ  
["major","mino  
jq
```

```
json 
```

```

{
  "conversatio
b908-5ed6c68fa
  "status": "S
  "response":
  {"major\":2,\
  "duration_se
  "num_turns":
  "structured_
    "major": 2
    "minor": 1
    "patch": 3
  },
  "json_schema
    "type": "o
    "propertie
      "major":
        "type"
      },
      "minor":
        "type"
    },
    "patch":
      "type"
    }
  },
  "required"
    "major",
    "minor",
    "patch"
  ]
},
"usage": {
  "input_tok
  "output_to
  "thinking_
  "cache_rea
  "total_tok
}
}

```

The flag accepts a
schema string, a

path to a `.json` schema file, or a primitive type name (`string`, `number`, `integer`, `boolean`). Read the parsed value from `structured_output`:

```
bash 
agy -p "Parse
version string
object with in
major, minor,
--output-for
--json-schem
'{"type":"obje
{"major":
{"type":"integ
{"type":"integ
{"type":"integ
["major","mino
| jq '.struc
```

Streaming JSON

Set

```
--output-format
stream-json
```

to emit one JSON object per line (NDJSON) as the run progresses. Use this format to observe tool calls and token usage in real time.

```
bash
agy -p "In
one
sentence,
what is a
git rebase?"
--output-
format
stream-json
```

The stream begins with one `init` event, followed by any number of `step_update` events, and ends with exactly one `result` event (the `cwd` and `tools` array are abbreviated below):

```
json
{"event":"init
{"cwd":"/home/
["ask_permissi
{"event":"step
058a026ef20a",
{"event":"step
058a026ef20a",
rewrites a bra
them onto a ne
{"input_tokens
{"event":"step
058a026ef20a",
{"input_tokens
{"event":"resu
058a026ef20a",
systematically
commit.\n","du
{"input_tokens
```

When the response streams in chunks, the `agent_response` step emits one or more `ACTIVE` events carrying partial `text_delta` fragments before its final `DONE` ; short responses like this one arrive in a single `DONE` .

Every line is an event object whose `event` field names

Products ▾Use Cases ▾PricingBlogResources ▾

Home

Antigravity 2.0 v2.4.3 >

Antigravity CLI v1.1.9 ▾

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

init

init

step_update

step_update

result

result

On this Page

Once, at Headless mode start

Run a single prompt

Output formats

For each step

Parse output with jq transition or text delta

Continue a conversation

Select a model, effort, or ag...

Once, at the end

Permissions in headless mode (same shape as json)

Handle exit codes and errors

Flag reference

Example: run the agent in CI

The `init` payload

https://antigravity.google/docs/cli/headless

Page 13 of 34

Headless mode		
Artifacts		>
Conversations		
Agent Capabilities		>
Projects		
Settings		>
AI Credits		
Customizations		>
Commands		>
Best Practices		
Troubleshooting		
Reference		
Antigravity SDK	v0.1.7	>
Antigravity IDE	v2.1.1	>

records the run configuration.

`model` and `agent` appear only when set with `--model` or `--agent` ; `permission_mode` is `request-review` by default (and `always-proceed` under `--dangerously-skip-permissions`):

Next steps

Field	Type	Description
<code>cwd</code>	string	Working directory
<code>tools</code>	string[]	Names of all available tools
<code>permission_mode</code>	string	Effective permission mode
<code>model</code>	string	Model in use, when overridden
<code>agent</code>	string	Active agent, when overridden
<code>json_schema</code>	object	Enforced schema, when set

with

--json-schema

Each `step_update` payload describes one step. Observed `step_type` values include `user_input` , `agent_response` , `tool` , and `checkpoint` ; `state` is `ACTIVE` while a step runs and `DONE` when it finishes:

Field	Type	Description
<code>conversation_id</code>	string	ID of the conversation
<code>step_index</code>	number	Zero-based index of the step
<code>state</code>	string	<code>ACTIVE</code> or <code>DONE</code>
<code>step_type</code>	string	Step category, for example <code>agent_response</code> or <code>tool</code>
<code>tool_name</code>	string	Canonical tool name, on tool steps
<code>text_delta</code>	string	Incremental

		response text
<code>duration_seconds</code>	number	Step duration, when known
<code>usage</code>	object	Per-step token usage, when known
<code>tool_info</code>	object	Tool invocation details (see below)
<code>subagent_info</code>	object	Subagent invocation details

Tool calls in the stream

On tool steps, `tool_info` carries the call and its result. This is a real tool step from a run that executed

```
echo
hello_headless_de
:
```

json 

```

{
  "event": "st
  "step_update
    "conversat
    "edb1c8c1-50ba
    412d0e9d47c3",
    "step_inde
    "state": "
    "step_type
    "tool_name
    "run_command",
    "duration_
    0.07,
    "tool_info
      "name":
    "run_command",
      "paramet
        "Comma
    "echo
    hello_headless
      },
      "output"
    "hello_headles
      }
    }
  }
}

```

`tool_info` holds
`name` ,
`parameters` ,
`output` , and —
 when the tool fails
 — an `error`
 object with `type`
 and `message` .
 Steps that spawn
 subagents carry
`subagent_info`
 instead, listing each
 subagent under
`subagents` (with
`type_name` , `role`

```
, conversation_id  
, log_uri , and  
workspace_uris ).
```

Structured output in the stream

With

```
--json-schema ,  
the schema applies  
to the terminal  
result event,  
which carries the  
same  
structured_output  
and json_schema  
fields as the json  
envelope.
```

Parse output with jq

stdout is
machine-readable,
so jq extracts
exactly what you
need.

Get the response
text from a JSON
run:

```
bash 
```

```
agy -p "Name
three
popular
version
control
systems,
comma-
separated."
--output-
format json
| jq -r
'.response'
```

```
text
Git,
Subversion,
Mercurial.
```

Concatenate
streaming text as it
arrives:

```
bash
agy -p "Explai
conflict is in
--output-forma
| jq -j
'select(.event
| .step_update
empty'
```

Read token usage
from the terminal

result event:

```
bash
```

```
agy -p "In one
what is a git
output-format
\
  | jq
'select(.event
| .result.usag
```

Tip: Use `jq -j` (join output) when concatenating `text_delta` fragments so `jq` does not insert newlines between them.

Continue a conversation

Headless runs are stateless by default. Resume prior context with `--continue ( -c )` for the most recent conversation, or `--conversation` with an ID from a previous run's `conversation_id` :

bash 

```
# Continue
the most
recent
conversation.
agy -p "Now
explain your
previous
answer in
more detail"
--continue
```

```
# Resume a
specific
conversation
by ID.
agy -p
"Summarize
what we
discussed" --
conversation
055a398f-
db14-4c5f-
abbb-
1bf03f8120a7
```

Select a model, effort, or agent

List the available
model slugs, then
pin one for the run:

```
bash   
agy models
```

```
text 
```

gemini-3.6-
flash-high
Gemini 3.6
Flash (High)
gemini-3.6-
flash-medium
Gemini 3.6
Flash
(Medium)
gemini-3.5-
flash-medium
Gemini 3.5
Flash
(Medium)
gemini-3.1-
pro-high
Gemini 3.1
Pro (High)
claude-
sonnet-4-6
Claude
Sonnet 4.6
(Thinking)
...

bash 

```
# Pin a model
by slug.
agy -p
"Reverse the
string
antigravity."
--model
gemini-3.5-
flash-medium
```

```
# Set
reasoning
effort (low,
medium, or
high).
agy -p
"Outline a
plan to add
caching to
this
service." --
effort high
```

```
# Select an
agent (list
them with
`agy
agents`).
agy -p
"Review this
function for
edge cases."
--agent
<agent-name>
```

Unlike the
interactive UI,
headless mode
does not silently fall
back when
`--model` names an
unknown model. It
exits non-zero with

an `ERROR` status
so a pinned pipeline
fails loudly instead
of running the
wrong model.

Permissions in headless mode

There is no
interactive prompt
in headless mode,
so tools that would
normally ask for
confirmation are
handled by policy.

By default, the CLI
respects the
permission mode in
your settings. A tool
that requires
approval it cannot
obtain is soft-
denied: the run
continues, exits `0`,
and prints a notice
to `stderr` naming
the tool and how to
allow it. Reading
and writing files
inside your active
workspace is auto-
allowed; actions
such as shell
commands default
to **Ask** and are soft-
denied in headless
mode unless you
grant them.

Grant a tool ahead
of time by adding
an

```
action(target)
```

rule under

```
permissions.allow
```

in

```
~/.gemini/antigra  
cli/settings.json
```

:

```
json
{
  "permissions": {
    "allow": [
      "command(git)",
      "command",
      "run",
      "(build|lint|te",
      "write_file(sr",
      ]
    }
  }
}
```

To auto-approve
every tool for a run,
pass

```
--dangerously-  
skip-permissions
```

:

```
bash
```

```
agy -p "Run
the test
suite and
report
failures" --
dangerously-
skip-
permissions
```

Warning:

```
--dangerously-
skip-permissions
```

approves all tool calls, including file writes and command execution. Prefer scoped

```
permissions.allow
```

rules unless you fully trust the prompt and environment. See [Permissions](#) for the full rule syntax.

Handle exit codes and errors

A successful run exits `0`. A run that fails to produce a response exits non-zero and writes the reason to `stderr`.

In `json` and `stream-json` modes, the failure also appears in the `status` and

`error` fields.

For example,
pinning an unknown
model exits `1` and
returns an error
envelope:

```
bash   
agy -p "hi"  
--model  
does-not-  
exist-model  
--output-  
format json;  
echo  
"exit=$?"
```

```
json 
```

```

{
  "conversatio
  "",
  "status": "E
  "response":
  "error": "in
model selectio
model \"does-n
exist-model\"
effort \"\")):
does-not-exist
is not recogni
a known model
custom model i
settings\nAvai
models:\n Gem
3.6 Flash (Hig
...\",

  "duration_seco
  0,
  "num_turns":
  "usage": {
    "input_tok
  0,
    "output_to
  0,

  "thinking_toke
  0,

  "cache_read_to
  0,
    "total_tok
  0
  }
}

```

```

text 
exit=1

```

The `status` field reports the terminal state of the run:

Status	Meaning
<code>SUCCESS</code>	The run completed and produced a response
<code>ERROR</code>	The run ended with an error
<code>CANCELED</code>	The run was canceled
<code>INTERRUPTED</code>	The run was interrupted (for example, <code>SIGINT</code>)
<code>INVALID</code>	The run reached an invalid state
<code>WAITING</code>	The run ended while waiting on input
<code>RUNNING</code>	The run did not reach a

terminal
state

By default, a run waits up to five minutes for a response. Adjust the ceiling with `--print-timeout :`

```
bash
agy -p
"Summarize
the design
tradeoffs of
optimistic
locking." --
print-
timeout 15m
```

Flag reference

Flag	Default	Description
<code>-p , --print , --prompt</code>	—	Run a single prompt non-interactively and print the response
<code>--output-format</code>	<code>text</code>	Output format: <code>text , json , or stream-json</code>

<code>--json-schema</code>	—	Schema string or file path to enforce structured output
<code>--model</code>	—	Model slug for this run (see <code>agy models</code>)
<code>--effort</code>	—	Reasoning effort: <code>low</code> , <code>medium</code> , or <code>high</code>
<code>--agent</code>	—	Agent for this run (see <code>agy agents</code>)
<code>--continue</code> , <code>-c</code>	<code>false</code>	Continue the most recent conversation
<code>--conversation</code>	—	Resume a conversation by ID
<code>--dangerously-skip-permissions</code>	<code>false</code>	Auto-approve all tool permission requests

<code>--print-timeout</code>	<code>5m</code>	Maximum time to wait for a response
<code>--sandbox</code>	<code>false</code>	Run with terminal sandbox restrictions enabled

Example: run the agent in CI

Fail the job on error and save the response:

bash 

```
#!/usr/bin/env
bash
set -euo
pipefail

result=$(agy -
p "Name three
popular
version
control
systems,
comma-
separated." \
--output-
format json \
--print-
timeout 10m)

status=$(echo
"$result" | jq
-r '.status')
if [[
"$status" !=
"SUCCESS" ]];
then
    echo "Agent
run failed:
$(echo
"$result" | jq
-r '.error')"
    >&2
    exit 1
fi

echo "$result"
| jq -r
'.response' >
result.txt
```

Next steps

- Prompting & Interaction:
Write effective prompts for the agent.
- Permissions:
Configure allow, deny, and ask rules.
- Background Tasks & Subagents:
Delegate work to specialized agents.
- Reference:
Full command and flag reference.

Prompting Reviewing >
< & Artifacts
Interaction